

Two Failure Modes, Not One: Extraction Error and Policy Misspecification in Static Analysis of Agent Workflows

Anonymous submission

Abstract

Static analysis of agent workflows raises overwhelmingly non-actionable findings, and the field attributes this to one cause—a lossy extractor. We show there are two, and that fixing the extractor addresses the smaller one. Mining 922 extracted file-level graphs from public GitHub across LangGraph, CrewAI, AutoGen, and Google ADK, we validate a 119-graph sample against LLM-reconstructed reference graphs (LLM-assisted exploratory labels, not human-validated ground truth). Triaging 186 flags attributes 42.5% to extraction artifacts but 50.0% to policy misspecification—checks firing correctly, on faithful graphs, against workflows their policy never governed. A symmetric ablation is decisive: the identical checks on near-oracle graphs remove only 62 of 184 flags, leaving ≥ 120 non-actionable findings on 119 workflows. The causes separate by check family—extraction explains 98.6% of non-actionable structural flags, policy specification 83.0% of human-gate flags, where a risk-aware policy moves firings from 113 to 0 without touching a graph. One pre-registered policy applied uniformly revises human-gate violations fourfold upward (3 to 12 of 119), three outside both original flag universes, and reveals that every workflow the checks cleared was cleared by a *placebo* gate—a node typed HUMAN whose body cannot block. The same ablation on *misses* inverts the budget: all 12 violations escape the as-mined pipeline, yet a near-oracle graph recovers one, the effect living inside a node body no such graph represents. We also give the static machinery two defect-independent jobs: provenance-gated certification returning *inconclusive* rather than a false *safe*, and a path-sensitive product that prunes runtime monitors soundly relative to an authored abstraction. The lesson is methodological: report extraction fidelity alongside every static-analysis claim about agent workflows.

1 Introduction

Large language models are increasingly deployed as autonomous agents that call external tools and APIs (Schick et al. 2023; Yao et al. 2023), and orchestration frameworks structure this behavior as explicit *workflow graphs*: LangGraph (LangChain 2025), CrewAI (CrewAI 2025), AutoGen (Wu et al. 2024), and Google ADK (Google 2025)

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

represent computation steps as nodes and allowable transitions as edges. Safety enforcement remains almost entirely a *runtime* concern: NeMo Guardrails (NVIDIA 2023) and LlamaGuard (Inan et al. 2023) filter individual calls at execution time, detecting a violation only when the offending path is exercised.

Some defects, however, are topological. Consider an email-triage agent whose developer adds a `draft_response` node but forgets to connect it to `send`: normal-priority emails silently halt at a dead end. A runtime guardrail cannot catch this unless that path runs, but a static analysis of the graph flags it and emits a witness trace. Classical model checkers (Clarke, Grumberg, and Peled 1999; Holzmann 1997; Cimatti et al. 2002) verify such properties, but require hand-translation into Promela or SMV. For conventional static analyzers, adoption is decided not by detection power but by the rate of actionable findings (Bessey et al. 2010; Johnson et al. 2013; Sadowski et al. 2018); the analogous question for agent workflows—how often these graphs contain the defects static checks target—has received far less measurement.

Thesis. Answering it requires a prior question: *can we trust the graph?* A static check is only as credible as the behavioral model an extractor recovers from source, and on public agent code that extraction is lossy—it manufactures false flags and hides true defects at once. That much is expected, and we quantify it; our central finding is that it is only *half* the story. Attributing every non-actionable flag to a cause, extraction artifacts account for 42.5% but **policy misspecification accounts for 50.0%**: checks firing correctly, on faithful graphs, against workflows their policy was never meant to govern. The decisive test is an ablation that holds the checks fixed and swaps in near-oracle graphs: it removes only 62 of 184 flags, leaving ≥ 120 non-actionable findings standing on 119 workflows. **A perfect extractor would not make these tools usable.** The causes separate cleanly by check family—extraction explains 98.6% of non-actionable structural flags, policy specification 83.0% of non-actionable human-gate flags (92% of attributed flags)—so the aggregate tilts check-ward only because human-gate checks are 61% of all flags. Static verification of agent workflows is thus a *measurement* problem with two error budgets, not one. The analyzer we measure with is prior infrastructure; this paper’s

contribution is not that tool but the cross-framework fidelity measurement it enables and the limits it exposes.

Study. We use it as a measurement instrument on 922 extracted file-level graphs mined from public GitHub across the four frameworks—a search-ranked convenience corpus, not a probability sample—and validate a quota-selected 119-graph sample against *LLM-reconstructed reference graphs* from an independent, adversarially cross-checked LLM pass. These are *LLM-assisted exploratory labels*, not human-validated ground truth; a two-human validation protocol (samples staged) is required future work, and we bound our claims accordingly.

Three estimands, not one prevalence. A pooled “prevalence” conflates distinct quantities, so we separate three (Table 2): flag-level precision; *structural-defect* prevalence, a topological property (1 of 119); and *human-gate policy-violation* prevalence, a normative judgment. One pre-registered policy applied uniformly, in place of an ad-hoc two-check procedure, revises the last fourfold upward, from 3 to **12 of 119** [5.9, 16.8]%, three outside *both* original flag universes—false negatives a flag-triage study structurally cannot see.

Contributions.

- **A two-cause attribution of non-actionable findings**, with the symmetric ablation the comparison requires, establishing a check-side error floor $\approx 1.9\times$ the extraction-attributable share and localizing each cause to a check family. A label-independent *survival test* reproduces it from graphs alone (93.0% agreement), separating checker error (1.1%) from policy misspecification (50.5%); run on *misses* it inverts the budget (Section 3).
- **A cross-framework extraction-fidelity measurement** (LLM-assisted), with a matched, provenance-disambiguated corrected-instrument ablation isolating extraction from check error.
- **A separation of three estimands** the field pools, the last re-measured under one uniformly applied pre-registered policy, which revises it fourfold upward and exposes *placebo* human gates biasing every reconstructed-graph estimand downward (Table 2).
- **A conservative certification gate** returning *inconclusive* rather than a false safe when extraction fails a provenance screen, plus a path-sensitive product pruning runtime monitors—soundly *relative to an authored abstraction*, since exact provenance screens for but does not prove event-trace conservatism (Sections 2, 4).

2 The Analyzer

Unified graph model. The analyzer represents a workflow as a directed graph $G = (V, E, v_0, V_f)$ with entry v_0 and exit set V_f . Each node carries a kind $\kappa_V(v)$ over ENTRY, EXIT, TOOL, LLM, ROUTER, HUMAN, PASSTHROUGH, SUBGRAPH and, for tool nodes, bound tool names; each edge is DIRECT, CONDITIONAL, PARALLEL, or LOOP. Every node and edge also carries extraction *provenance*—an origin (runtime object,

explicit AST, inferred, synthesized, unknown), a confidence (exact, may, heuristic), and a source span—so structure read from a declaration is distinguishable from structure the extractor guessed, and witness traces inherit the weakest confidence they traverse. Per-framework extractors bridge four representations into this model by either of two paths: a run-time extractor reading the compiled graph object, or a static AST fallback for large-scale mining, where installing every repository’s dependencies is infeasible.

An orthogonal attribute ontology. The single-label kinds above are a compatibility view: forcing each element into one bucket mislabels multi-faceted nodes—an LLM node that also invokes a tool—and was itself a source of extraction error. We therefore layer a non-exclusive ontology over κ_V/κ_E (vocabulary in the supplement): each node carries an *effect* set (read, write, execute, communicate, financial, delete) and a *capability* set, and each edge a *back-edge* flag. It is primary in the data model, but the per-framework extractors do *not* yet populate these fields: every mined graph carries defaults, and the sets are assigned at corpus-build time by a framework-independent rule over node kinds and declared tool names. The ontology therefore inherits the same declared-tool blindness as the risk-aware gate (Section 3) rather than curing it.

Structural checks. Six checks run over the topology, each in $O(|V| + |E|)$ and each emitting a *witness trace* (Clarke, Grumberg, and Peled 1999) on failure: *exit reachability* ($V_f \subseteq \text{Reach}(v_0)$), *reverse reachability* (reachable nodes with no path to an exit), *dead-end detection* (non-exit nodes with no successor), *router shape* (router outgoing edges must be conditional), *human presence* (no HUMAN node—a deliberately blunt policy check we evaluate against a risk-aware variant), and *tool declaration* (tool nodes with an empty tool set). A seventh, path-sensitive check—*human-gate coverage*—flags a workflow only if a *sensitive* tool is reachable from v_0 without passing a HUMAN node (Section 3).

Temporal policies with LTL_f semantics. It also supports a policy DSL over finite traces (LTL_f (De Giacomo and Vardi 2013)): an execution is a finite event sequence, and each of seven DSL forms denotes an LTL_f formula—forbidden ($G \neg a$), response ($G(a \rightarrow F b)$), strong until, bounded response ($G(a \rightarrow F_{\leq k} b)$), response chain, conjunction, disjunction (grammar in the supplement). Formulas compile compositionally to a DFA by formula progression (2–3 states for base patterns, $k+2$ for bounded response), with two disjoint violation mechanisms: *violation states* are exactly the doomed states, so a bad prefix is flagged at the earliest event making every extension violating; and a pending F-obligation violates at termination. The compiler is differentially tested against an independent reference evaluator, exhaustively to length six for two-atom formulas. Runtime monitors evaluate events in $O(1)$; obligations open when a run aborts are reported inconclusive (Bauer, Leucker, and Schallhart 2011).

Static temporal verification. Policies are checked statically via a graph $\times$ DFA product: BFS from (v_0, q_0) advances the DFA at each node and follows graph edges, modeling executions as finite *maximal* paths. Because static extraction

cannot prove which of a tool node’s declared tools will be invoked, or whether any will be, the product is *conservative*: at each tool node it closes the DFA state over every finite invocation sequence drawn from the declared tools, including the empty one—a false alarm is acceptable, a false proof is not. The checker returns *may-violate* (some resolution reaches a bad prefix or a pending obligation at termination), *safe* (none does), or *inconclusive* (a reachable product cycle postpones an obligation forever, or no termination point is reachable; an engineering-scope choice, not undecidability). Explored states are bounded by $|V| \cdot |Q|$.

Soundness gated on certified extraction. Soundness is a claim about *event* traces, not graph paths. Let $\text{Traces}_{\text{ev}}(P)$ be the concrete finite terminating event traces a workflow P can emit, and let λ label each event, inducing a graph event-language $\mathcal{L}(G, \lambda)$. If $\text{Traces}_{\text{ev}}(P) \subseteq \mathcal{L}(G, \lambda)$ (*event-trace conservatism*) and every trace in $\mathcal{L}(G, \lambda)$ is accepted by the policy DFA, then every concrete finite terminating execution of P satisfies the policy—the guarantee we exploit for monitor pruning (Section 4); the proof is in the supplement. Graph-path containment alone does *not* suffice: a node may issue tool calls the graph never records. Because a lossy extractor cannot be assumed event-trace conservative, the checker returns *safe* only when the traversed region is *certified*—every traversed edge and every visited tool-node binding carries *exact* provenance, or the caller asserts an authored graph—stamping the result with a *certified* key; otherwise it downgrades to *inconclusive* with reason *uncertified_extraction*. A *may-violate* finding is *never* suppressed by the gate.

What the gate does and does not certify. Exact provenance describes *how an element was obtained*—read from a declaration rather than guessed—and is *not* equivalent to event-trace conservatism. The two come apart exactly where our measurements say they do: an edge can be read exactly while tool calls inside a node’s body go unmodeled, the `tool`→`llm` kind error that dominates LangGraph’s residual error and hides side-effecting tools from the risk-aware gate in 32 of 119 sampled workflows (Section 3); nothing in the gate inspects node bodies for unrecorded events. It therefore certifies *provenance metadata, or a caller’s assertion*—not the containment $\text{Traces}_{\text{ev}}(P) \subseteq \mathcal{L}(G, \lambda)$ the soundness theorem requires. We accordingly make the weaker claim the machinery supports: the product is sound *relative to an authored abstraction* whose event coverage the caller asserts, with exact provenance a necessary screening condition, not a proof. Independent event-coverage analysis is future work. On mined graphs the gate withholds *safe* outright, so no false proof is emitted.

3 Base-Rate Study on Public GitHub Workflows

To estimate defect base rates on independently authored workflows—and measure extractor fidelity on code we did not write—we mined public GitHub across all four frameworks. The population is *public GitHub files matching framework-specific code searches*, not production deploy-

ments; the unit is one source file mapped to one *extracted file-level graph*, so a fully executable-workflow-root unit is future work. All prevalence figures are *LLM-assisted exploratory estimates*, not ground truth.

Mining and corpus census. We mined GitHub on 2026-07-11 with eleven `gh search code` queries (three per framework, two for CrewAI), retaining the top **40** results per query in GitHub’s default relevance order and cloning the first **180** unique repositories in that order; within a cloned repository every `.py` file was extracted (no per-file cap). After removing one repository of our own test fixtures (8 files; a contamination detected during revision and excluded from every number), the corpus is **922 extracted file-level graphs** from 252 distinct repositories (LangGraph 400, AutoGen 359, CrewAI 113, ADK 50), with provenance recorded as repository, commit SHA, and file path. This is a *search-ranked convenience corpus*, not a probability sample: the ranking function is proprietary and non-stationary and the index denominator unpublished, so inclusion probabilities are *undefined* rather than merely unestimated. Every rate is a property of this corpus and the instrument applied to it; we make no claim of the form “ $X\%$ of agent workflows on GitHub.” Mined graphs are small (median 2 non-sentinel nodes, mean 3.9) and predominantly static linear pipelines, and the corpus is *not* mostly applications: 32% of files carry tutorial/test/demo names, 170 graphs are topological duplicates, and a source-level classification of the validation sample found 44% tutorial, 17% toy demo, 7% test, and 33% application-like code.

Reference graphs and validation. On a 119-graph validation sample from 87 repositories (LangGraph 40, CrewAI 20, AutoGen 35, ADK 24) we ran two independent LLM-agent passes (Claude Opus 4.8 under a multi-agent harness; all outputs released). First, an agent reconstructed a reference graph from source alone, blind to the extractor’s output and following its node-id conventions; we score node/edge precision–recall and node-kind accuracy against these *LLM-reconstructed reference graphs* (confidence: 77 high, 33 medium, 9 low). Second, every flag was triaged from source (`REAL/EXTRACTION-ARTIFACT/INTENTIONAL`) and an adversarial verifier attempted to overturn each label (4.6% overturned; unresolved labels recorded as `ARGUABLE`). Reference graphs are LLM reconstructions, not human annotations, and both passes share a model family. Results are stable on the high-confidence subset, every triage label carries a source-line justification, and a re-audit overturned one label of 16 re-examined. **Two-annotator independent human validation was not performed; it is required future work, and the protocol and samples are staged.** We report Wilson 95% CIs and repository-clustered bootstrap CIs (10,000 resamples), since workflows from one repository are not independent.

How the validation sample was selected. The 119 graphs were chosen under fixed per-framework quotas *without randomization*: the identifiers exist only as literal lists in the released harness scripts, with no seed or reproducible selection rule, so we do not claim the sample is random within frame-

Table 1: Extractor fidelity on the *matched, provenance-disambiguated* set ($n=106$): both instruments scored on identical graphs, colliding records excluded. Edge P and kind are identical between instruments except where shown—the corrected instrument changes *exactly one* framework.

Framework	n	Edge P	Edge R _{v1}	Edge R _{v2}	Kind
LangGraph	38	0.954	0.678	0.836	0.659/0.670
CrewAI	20	0.700	0.692	0.692	0.929
AutoGen	30	0.966	0.778	0.778	0.890
ADK	18	0.394	0.343	0.343	0.920
Overall	106	0.814	0.652	0.709	0.820/0.824

work, nor call it stratified. The framework-share reweighting is consequently a *corpus-share-weighted descriptive statistic*—the rate the corpus would show if each framework’s sampled rate held across its members, correcting the sample’s $\sim 5\times$ over-representation of ADK—not a design-based estimate, and the intervals quantify sampling noise *within this corpus* only. We apply no finite-population correction, which would presuppose the simple random sampling this design does not provide.

A keying defect. We report an instrumentation error found while preparing this comparison, because it changes the numbers. Graphs were keyed by repository plus file *basename*, which is not unique: **76** slugs collide, and for those the two instruments scored *different source files* against the same reference graph. No colliding pair is repairable (supplement), so every colliding graph is dropped: **9** inside the fidelity set, cutting it from 115 matched graphs to the $n=106$ of Table 1, with 4 more of the 119 sampled unavailable at re-mining. The cost is not uniform—ADK loses 5 of 23 versus 0–6% elsewhere—so ADK’s row is the least stable. The keying is fixed in the released instrument.

Extraction fidelity is the bottleneck—for structural checks. Node detection stays strong and stable across instruments (v2 overall $P=0.914$, $R=0.868$; v1 0.914/0.862), but edge recall and node-kind accuracy are the bottleneck and are strongly framework-dependent (Table 1): the corrected instrument recovers only 0.34 of ADK’s edges—whose nested SequentialAgent/ParallelAgent composition, and AutoGen’s participant lists with dynamic speaker selection, encode topology *implicitly*—versus 0.84 on LangGraph’s explicit edges. On LangGraph the dominant remaining error is instead node-kind (`tool`→`llm`: nodes invoking tools in their body with no declared binding; kind accuracy 0.67), the direct cause of the risk-aware gate’s blindness below. This is a property of the static AST fallback, not of the runtime extractors (for ADK the runtime graph is far richer, 16 vs. ~ 5 edges); realizing that ceiling would require sandboxed execution of untrusted code, which we do not attempt.

A corrected-instrument ablation demonstrates the diagnosis *causally* for the structural checks. The correction bundles *three* behavioral changes—a LangGraph three-argument `add_conditional_edges` fix, a LangGraph declared-tool metadata fix, and ADK LoopAgent support—but, re-extracting at identical pinned commits with the check algo-

rithms untouched, only *one* has a measurable effect: LangGraph edge recall rises $0.678 \rightarrow 0.836$, and of the 54 true edges newly recovered, **54/54** are targets of a three-argument `add_conditional_edges` path map across 24 call sites. The other three frameworks are bit-identical between instruments (Table 1), so LoopAgent cannot be credited with an improvement. On the matched collision-free corpus ($n=853$) the correction removes roughly a quarter of the structural flags ($299 \rightarrow 229$ flagged workflows; unreachable-exit flags $81 \rightarrow 29$), confirming extraction as the binding constraint *for the structural checks*—a scope the attribution below makes precise, and which does not extend to the policy checks. Both instruments are released.

Flag precision on extracted graphs. Distinct from prevalence, what fraction of the checker’s *flags* is genuine? The as-mined study raised 186 flags on 114 file-level graphs; adversarial triage confirmed 2 as genuine (both missing human gates), classifying 42% as extraction artifacts and 50% as intentional design—a workflow-level PPV of $2/114 = 1.75\%$ [0.48, 6.17]% and a flag-level PPV of $2/186 = 1.08\%$ [0.30, 3.84]%. Deduping correlated flags to root cause ($186 \rightarrow 161$) gives 1.24%, and counting all 12 ARGUABLE labels pessimistically bounds the rate at 7.53%. *None of the 72 structural flags was confirmed genuine* (CI [0, 5.1]%), and structural flags are almost entirely a LangGraph phenomenon—partly *by construction*, since the other three extractors emit connected topologies and cannot produce a dead end or unreachable exit. This as-mined universe is *distinct* from the reconstructed-graph re-run below; the two are never pooled.

Two failure modes. A precision number says findings are non-actionable but not *why*, and the answer decides what to fix. We attribute each of the 186 flags to one of four causes: EXTRACTOR (the graph misrepresents the code); CHECKER (check logic wrong given a correct graph); POLICY-SPEC (graph and check both correct, but the policy does not apply—a blunt human-presence requirement on a read-only tutorial); and LABEL (disputed judgment). The result does not support one cause: extraction artifacts account for 42.5% of flags [35.6, 49.7], policy misspecification for 50.0% [42.9, 57.1]—overlapping intervals in which extraction is, if anything, the *smaller* term.

Because the triage vocabulary asked “is this a real defect?”, not “whose fault is this flag?”, we separate CHECKER from POLICY-SPEC *mechanically*, with a survival test: re-run each check on the reference graph. A flag the near-oracle removes is EXTRACTOR; a *structural* flag that survives on a faithful graph yet is still not a defect indicts the check’s decision procedure (CHECKER); a surviving *policy* flag indicts its specification. Where the reference graph is itself known wrong—a GT-ERROR label, or a HUMAN node the placebo audit found non-discharging—we record REFERENCE-ERROR rather than force a side. This label-independent instrument agrees with the label-based one on **173 of 186** flags (93.0%) and closes the gap the earlier vocabulary left open: CHECKER is 1.1% [0.3, 3.8] against POLICY-SPEC 50.5%, with EXTRACTOR 38.2% and REFERENCE-ERROR 2.7%. Check-side error is almost entirely *specification*, not *logic*.

The decisive test. Attribution rests on labels, so we ran the symmetric ablation: hold the checks fixed, vary only the graph, using the 119 reference graphs as a near-oracle extractor. Flags fall from 184 to 122—an oracle removes **62 flags, roughly a third**, leaving ≥ 120 **non-actionable findings standing** on 119 workflows, a check-side floor $\approx 1.9\times$ the extraction-attributable share. The other knob is far more effective: holding the graph fixed and swapping the blunt policy for the risk-aware gate moves firings from **113 to 0**. The causes separate by check family: among non-actionable *structural* flags ($n=72$), 98.6% [92.5, 99.8] are extraction-attributable and 0% [0, 5.1] check-side, whereas among non-actionable *human-gate* flags ($n=112$) only 7.1% [3.7, 13.5] are extraction-attributable against 83.0% [75.0, 88.9] check-side—92% of attributed flags, the rest disputed labels. Because human-gate checks are 61% of all flags, the aggregate goes against a pure extraction story even though extraction dominates the structural checks. Better extractors fix reachability checks; only per-workflow policy instantiation fixes safety checks.

The same test, run on misses. Everything above decomposes false *positives*. Flag triage cannot see misses, so we run the ablation in the other direction, over the 12 confirmed HGP-1 violations. The as-mined risk-aware check reports **none** of them, and the cause is unmixed: no mined graph declares a sensitive tool at all, so all 12 misses are extraction (100% [75.7, 100]). Replacing the extractor with the reference graph—the same near-oracle swap—recovers only **1 of 12**. Of the 11 still missed, **9** have the effect absent even from the *reconstruction*, because it lives in a node *body* that the graph abstraction does not represent; 1 is a reconstruction that dropped statically declarable tools; 1 is a workflow the check cleared on a gate that cannot block. The two directions therefore have opposite error budgets: non-actionable findings are check-side, but misses are model-side—and a near-oracle graph repairs almost none of them, because the missing information sits *below* the abstraction rather than above it—the same boundary the certification gate runs into (Section 2).

Defining a defect. We re-ran every check on the 119 reference graphs—which can express dead ends and unreachable exits for all four frameworks—and adversarially triaged the results, recovering two genuine defects the extracted study missed. A *structural defect* is a topology error; a *human-gate policy violation* is a normative judgment that a side-effecting tool is reachable with no intervening HUMAN checkpoint, which a permissive policy would not yield. We report each as a **distinct estimand**, their union only as a secondary summary.

Three estimands and reweighting. Table 2 reports all three under the uniform policy. The one structural defect is a router mis-wiring in an ADK SQL agent, 1/119 [0.15, 4.61]%; the human-gate estimand is the re-audited 12/119 [5.86, 16.80]%; and the composite unions them—the structural case is ARGUABLE, not a violation, under HGP-1, so the union is a strict addition—giving 13/119 [6.50, 17.80]%. Per-framework violations run LangGraph 6/40, AutoGen

Table 2: Three defect estimands on the 119-graph validation sample (LLM-assisted exploratory estimates), with Wilson 95% CIs (%) and corpus-share-weighted estimates (clustered bootstrap, no FPC). The human-gate row is the uniform HGP-1 re-audit, superseding the 3/119 a two-check procedure reported.

Estimand	Sample	Wilson 95%	Weighted
Structural defect	1/119 (0.84%)	[0.15, 4.61]	0.23% [0.00, 0.80]
Human-gate (HGP-1)	12/119 (10.1%)	[5.86, 16.80]	12.30% [4.70, 21.34]
+ ARGUABLE	23/119 (19.3%)	[13.24, 27.34]	17.74% [9.52, 27.01]
Composite [†]	13/119 (10.9%)	[6.50, 17.80]	12.52% [4.93, 21.56]

[†]Heterogeneous secondary summary (union of the two primary estimands).

5/35, CrewAI 0/20, ADK 1/24. Reweighting each stratum to its corpus share (clustered bootstrap, no finite-population correction) *raises* rather than lowers these estimates, because LangGraph and AutoGen carry both the highest violation rates and 82% of corpus weight: human-gate 12.30% [4.70, 21.34], composite 12.52% [4.93, 21.56]. The weakest link is CrewAI’s 0%, resting on a single audited workflow.

The revision is *not* additive. Of the three cases the two-check procedure reported, HGP-1 retains only one; the other two become ARGUABLE, their side-effecting callees living in modules the single-file mining unit never captured. The uniform policy thus retains 1 of 3 and finds 11 new violations.

One policy, applied uniformly. The three policy cases above came from *two* different checks (blunt `human_presence` and path-sensitive `human_gate_coverage`) over two graph populations. That is not one estimand, and it cannot see a violation neither check flags. We therefore pre-registered a single operational policy (HGP-1; in the supplement) and applied it uniformly to every sampled workflow with a side-effecting tool at *source* level—all 32—regardless of flag universe. HGP-1 fixes the side-effecting categories by what the callee does in source rather than by name; requires a gate to be an explicit checkpoint or a blocking confirmation whose negative branch skips the effect, and to *dominate* the call site; and records undecidable cases as ARGUABLE, never silently as compliant. Demo, tutorial, and test status are *not* exemptions; genuine mocks and visibly sandboxed effects are.

The audit returns **12 violations, 9 compliant, 11 arguable**: 12/119 = 10.1% [5.9, 16.8] against the reported 3/119—a fourfold revision whose interval excludes the old estimate; ARGUABLE counted pessimistically bounds it at 23/119. The uncertainty points upward: 12/119 is a *lower bound*, since several ARGUABLE cases (a payment pipeline whose prompts forbid confirmation four times over) would likely resolve toward violation given repository-level rather than single-file source. It is a lower bound for a second reason we state plainly: only the 32 were read *under HGP-1*; the other 87 enter the denominator as non-violations by construction, having been excluded by the same source-level classification pass that produced the 32. An independent sweep of all 87 sources for effect signatures flagged 10 for manual

re-reading: 9 are correctly screened and 1 is borderline—an MCP workbench whose server module lies outside the mining unit—which HGP-1 would record as ARGUABLE, not a violation. No missed violation was found, so the residual is bounded by one ARGUABLE case. Three of the 12 lie *outside both flag universes*—no triage entry, neither check fired—confirming that false negatives exist beyond the union of the flag sets, where a flag-triage study can never find them.

Placebo gates: the reference graphs overstate oversight. The most consequential finding is not a count. All seven side-effecting workflows that *both* checks cleared were cleared because the reference graph held a node typed HUMAN—and on source inspection *none is a genuine dominating gate*: five are placebo or inert, one does not dominate the call site, one is unverifiable. One exemplar declares six `*_human_approval` nodes whose body reads a constructor flag hard-set to `True`: a checkpoint that cannot block, typed HUMAN by both extractor and reconstruction because it is named and shaped like one. Reference graphs assign HUMAN from naming and framework type, not from whether the body can block, so *every* human-gate estimand computed over them—including the risk-aware one—is systematically biased **downward**. The fidelity metric compounds it: for two of these workflows it records human-node recall 1.0, crediting the extractor with recovering oversight that does not exist. A near-oracle graph is not a semantic oracle.

Risk-aware gating. The blunt human-presence check fires on 91% of mined graphs with only 2 of 113 firings genuine—uninformative as a detector, motivating the risk-aware *human-gate coverage* check: flag only when a sensitive tool (destructive/write/exec/communication/financial, by a keyword lexicon over bound tool names) is reachable without a HUMAN gate. On the 922 mined graphs it fires zero times—a statement about extracted *declarations*, not the code: no mined graph declares a sensitive binding, while source review finds side-effecting tools in 32 of 119 sampled workflows, invisible through the `tool→llm` kind error. On reconstructed graphs it fires three times. Its recall is bounded by extraction fidelity, its precision—per the audit above—by whether a HUMAN node can block.

Threats to validity. *Construct*: the composite unions a structural property with a policy judgment, so we report the two separately. *Internal and label reliability*: reference graphs and triage labels come from LLM passes sharing a model family, so correlated errors are possible; confidence stratification, adversarial overturn rates, and a re-audit mitigate but do not replace the independent human judgment that has not been performed—a two-annotator protocol with staged samples is future work. *External*: the population is public GitHub files and the unit a file-level graph, so results may not transfer to executable roots or private code. *Sampling*: the frame is a convenience corpus with undefined inclusion probabilities, quota-selected without randomization. *Reference-graph semantics*: node kinds are assigned from naming, not from whether a body has the effect its kind implies—the placebo-gate audit shows this biases human-gate estimands downward. Every prevalence number is ac-

cordingly an LLM-assisted exploratory estimate, not ground truth.

4 Controlled Evaluation and Monitor Selection

Controlled detection test. A curated corpus of 18 workflows across all four frameworks, seeded with known anti-patterns, confirms the checks fire when defects are present: the analyzer detects every injected structural defect and, with a risk-aware sensitive-tool set, flags 8 workflows whose sensitive path is reachable without a human gate, versus 10 for the blunt check—declining to flag two that lack a human node but perform no sensitive action. This is a controlled test that the algorithms work, not evidence of prevalence. Verification is sub-second on synthetic graphs of 5,000 nodes; the supplement compares Promela/SMV modeling effort.

Static monitor selection. The graph $\times$ DFA product gives the static machinery a job independent of whether defects exist: deciding which runtime monitors to deploy. Its value is *soundness*, not scale, and it rests on the certification gate of Section 2, so every pruning number is sound *relative to an authored abstraction*—we assume, and do not prove, that a curated workflow’s nodes emit no events outside their declared bindings. A policy whose product is safe can never fire at runtime, so its monitor need not be deployed; `may_violate` is never suppressed. On the curated corpus the gate certifies 237 of 270 monitor instances (87.8%) as provably inert and declines 14 (all on one seeded-livelock workflow) as *inconclusive*, leaving 19 to runtime enforcement.

Descriptive vs. sound pruning on mined code. This rate does not survive lossy extraction. A $\sim 93\%$ “pruning” rate is available on the mined graphs, but it is pure *alphabet-inertness*: all 13,830 mined (workflow, policy) instances are inert only because no mined workflow invokes any of the 15 policies’ tools—a mismatched vocabulary, not a sound prune. On a may-provenance graph a missing edge could hide exactly the tool that makes a live monitor look inert, so the gate certifies *none* of them, refusing all as uncertified extraction.

Does path-sensitivity buy anything? Alphabet-inertness needs no product; the product earns its keep only when a policy’s atoms occur and inertness turns on *which* paths reach them. On 18 (workflow, policy) pairs of that kind, a runtime-only strategy certifies 0, since one unobserved path forbids the claim; alphabet-only prunes 1, node-reachability 3, and the path-sensitive product 10—with no false prunes across 50 concrete executions. We are deliberately conservative: five of the 9 incremental cases are linear response-obligation pipelines in which the consequent trivially dominates every path, and the branching and looping evidence rests on 4 purpose-built *synthetic* fixtures. Fifty executions cannot establish the absence of false prunes; this is a mechanism demonstration, not evidence of scale.

5 Related Work

Runtime agent safety. NeMo Guardrails (NVIDIA 2023), Guardrails AI (Guardrails AI 2023), LlamaGuard (Inan et al.

2023), and Rebuff (Protectai 2023) enforce safety at the point of LLM output or tool call, catching violations only when the offending path runs; ToolEmu (Ruan et al. 2024) and AgentDojo (Debenedetti et al. 2024) measure agent risk by executing behaviors. Our analyzer is complementary: it analyzes topology before deployment and decides which runtime monitors are needed (Section 4).

Concurrent agent-policy systems. A concurrent line of work enforces policies at the tool-call boundary with formal machinery: Agent-C compiles temporal safety specifications to constraints checked at generation time with conformance guarantees (Kamath et al. 2026); AgentSpec provides a runtime-enforcement DSL (Wang, Poskitt, and Sun 2025); ClawGuard and SecureClaw interpose rule sets and authorization boundaries against prompt injection (Zhao et al. 2026; Ma and Schmid 2026); PolicyGuard trains a guardrail model for trajectory-level violation detection (Wen et al. 2025); and AgentFlow builds cross-framework agent dependency graphs for static security analysis at scale (Wang et al. 2026). Two bound our contribution sharply. Agent-C already enforces temporal specifications with conformance guarantees, so we claim no novelty for temporal enforcement; AgentFlow already builds richer cross-framework dependency graphs over 5,399 programs, so we claim none for cross-framework graph construction or scale. What remains ours is the measurement neither performs: *how faithfully* such extracted models correspond to the code they abstract. AgentFlow reports no validation of its graphs against reference models; our result is that on public code this is not a formality but a dominant term in the error budget.

Empirical characterizations of agent defects. Recent studies catalogue where agent systems break: a defect taxonomy from developer discussions (Ning et al. 2024), nine root causes across $\sim 1,026$ orchestration bugs (Xue et al. 2025), and 221 vulnerabilities spanning fourteen types (Shen et al. 2025). We ask not *which* defects occur, but *how reliably a static analyzer can recover the behavioral model it needs to find them*.

Model checking and workflow soundness. Classical model checkers—SPIN (Holzmann 1997), NuSMV (Cimatti et al. 2002), CBMC (Kroening and Tautschnig 2014)—verify the same class of properties but require manual translation into Promela/SMV/annotated C. Our policy semantics follow LTL on finite traces (De Giacomo and Vardi 2013), with monitoring in the style of JavaMOP (Jin et al. 2012) and three-valued runtime verification (Bauer, Leucker, and Schallhart 2011). Business-process soundness for Petri nets (van der Aalst 2011) and BPMN (Dijkman, Dumas, and Ouyang 2008) studies analogous reachability properties over visual notations rather than the programmatic APIs agent frameworks expose.

Base rates for static analysis. Industrial experience established that false-positive and actionability rates, not detector cleverness, decide adoption (Bessey et al. 2010; Johnson et al. 2013; Sadowski et al. 2018). Ours is the agent-workflow instance of that genre, and adds a failure mode those studies did not face: the *model extraction* step carries its own large

share of the error budget, so non-actionability must be apportioned between extractor and policy rather than attributed wholly to the checks. Broader agent-safety work emphasizes alignment, capability evaluation, and governance (Anthropic 2023; METR 2024; Gabriel et al. 2024); our contribution is orthogonal and empirical.

6 Conclusion

Static checks on agent workflows are only as credible as the behavioral model they run on—but repairing that model is only half the problem. On 922 extracted file-level graphs from 252 public GitHub repositories, non-actionable findings divide roughly evenly between extraction error (42.5%) and policy misspecification (50.0%), and near-oracle graphs remove only 62 of 184 flags: a perfect extractor would not make these tools usable. The causes localize cleanly—extraction explains 98.6% of non-actionable structural flags, policy specification 83.0% of human-gate flags.

The estimates are textured, not a null. Structural defects sit at 0.84% of the validation sample; one pre-registered policy applied uniformly puts human-gate violations at 10.1%, four-fold above an ad-hoc two-check procedure, with three outside both flag universes (Table 2). Every workflow both checks cleared was cleared by a *placebo* gate, so reference graphs—which assign kinds from naming, not behavior—overstate oversight and bias the estimand downward. Flag-level precision stays low (workflow-level PPV 1.8%), no structural flag on an extracted graph was a confirmed defect, and fidelity is poor: edge recall 0.71 overall, 0.34 on implicit-topology ADK. The static machinery keeps a defect-independent role: on authored graphs whose event coverage the caller asserts, the graph $\times$ DFA product prunes 237 of 270 curated monitor instances (88%)—sound relative to that abstraction, not a proof it is event-complete.

Limitations. The population is public GitHub, not production systems, and two thirds is tutorial-grade; the unit is an extracted file-level graph, not an executable workflow root. Reference graphs and triage labels are LLM reconstructions with adversarial verification, not human annotations, and a two-human independent validation is the immediate next step. CIs are wide at $n=119$. We read these results as a redirection toward higher-fidelity extraction *and* per-workflow policy instantiation, neither alone.

Artifact. An anonymized artifact is submitted, not promised: the tool, both corpora, the mining and validation pipeline, reference graphs, justified triage labels, and all evaluation scripts.

Ethical Statement

We mine only *public* GitHub repositories, record exact provenance (repository at commit SHA plus file path), and redistribute only derived graph abstractions, never third-party source; no personal data is processed. Confirmed defects are disclosed to the affected repositories before publication, without naming maintainers.

References

- Anthropic. 2023. Anthropic’s Responsible Scaling Policy. <https://www.anthropic.com/responsible-scaling-policy>. Accessed: 2026-03-01.
- Bauer, A.; Leucker, M.; and Schallhart, C. 2011. Runtime Verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4).
- Bessey, A.; Block, K.; Chelf, B.; Chou, A.; Fulton, B.; Hallem, S.; Henri-Gros, C.; Kamsky, A.; McPeak, S.; and Engler, D. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Communications of the ACM*, 53(2): 66–75.
- Cimatti, A.; Clarke, E.; Giunchiglia, E.; et al. 2002. NuSMV 2: An OpenSource Tool for Symbolic Model Checking. In *International Conference on Computer Aided Verification (CAV)*. Springer.
- Clarke, E. M.; Grumberg, O.; and Peled, D. A. 1999. *Model Checking*. MIT Press.
- CrewAI. 2025. CrewAI documentation. <https://docs.crewai.com>. Accessed: 2026-02-28.
- De Giacomo, G.; and Vardi, M. Y. 2013. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In *International Joint Conference on Artificial Intelligence (IJCAI)*.
- Debenedetti, E.; Zhang, J.; Balunović, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- Dijkman, R. M.; Dumas, M.; and Ouyang, C. 2008. Semantics and analysis of business process models in BPMN. *Information and Software Technology*, 50(12): 1281–1294.
- Gabriel, I.; et al. 2024. The Ethics of Advanced AI Assistants. *arXiv preprint arXiv:2404.16244*.
- Google. 2025. Google Agent Development Kit (ADK) documentation. <https://google.github.io/adk-docs/>. Accessed: 2026-02-28.
- Guardrails AI. 2023. Guardrails AI: Adding Guardrails to Large Language Models. <https://github.com/guardrails-ai/guardrails>. Accessed: 2026-03-01.
- Holzmann, G. J. 1997. The Model Checker SPIN. *IEEE Transactions on Software Engineering*, 23(5): 279–295.
- Inan, H.; Upasani, K.; Chi, J.; et al. 2023. LlamaGuard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674*.
- Jin, D.; Meredith, P. O.; Lee, C.; and Roşu, G. 2012. JavaMOP: Efficient Parametric Runtime Monitoring Framework. In *International Conference on Software Engineering (ICSE)*.
- Johnson, B.; Song, Y.; Murphy-Hill, E.; and Bowdidge, R. 2013. Why Don’t Software Developers Use Static Analysis Tools to Find Bugs? In *International Conference on Software Engineering (ICSE)*.
- Kamath, A.; Zhang, S.; Xu, C.; Ugare, S.; Singh, G.; and Misailovic, S. 2026. Enforcing Temporal Constraints for LLM Agents. Preprint. <https://github.com/structuredllm/agent-c>.
- Kroening, D.; and Tautschnig, M. 2014. CBMC – C Bounded Model Checker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer.
- LangChain. 2025. LangGraph documentation. <https://docs.langchain.com/oss/python/langgraph/overview>. Accessed: 2026-02-28.
- Ma, Y.; and Schmid, S. 2026. SecureClaw: Clawing Back Control of LLM Agents. *arXiv:2606.09549*.
- METR. 2024. METR: Model Evaluation and Threat Research. <https://metr.org>. Accessed: 2026-03-01.
- Ning, K.; Chen, J.; Zhang, J.; Li, W.; Wang, Z.; Feng, Y.; Zhang, W.; and Zheng, Z. 2024. Defining and Detecting the Defects of the Large Language Model-based Autonomous Agents. *arXiv preprint arXiv:2412.18371*.
- NVIDIA. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. <https://github.com/NVIDIA/NeMo-Guardrails>. Accessed: 2026-03-01.
- Protectai. 2023. Rebuff: Self-hardening Prompt Injection Detector. <https://github.com/protectai/rebuff>. Accessed: 2026-03-01.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations (ICLR)*.
- Sadowski, C.; Aftandilian, E.; Eagle, A.; Miller-Cushon, L.; and Jaspan, C. 2018. Lessons from Building Static Analysis Tools at Google. *Communications of the ACM*, 61(4): 58–66.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.
- Shen, Z.; Dai, J.; Zhang, Y.; and Yang, M. 2025. Security Debt in LLM Agent Applications: A Measurement Study of Vulnerabilities and Mitigation Trade-offs. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- van der Aalst, W. M. P. 2011. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer.
- Wang, H.; Poskitt, C. M.; and Sun, J. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. *arXiv preprint arXiv:2503.18666*.
- Wang, S.; Hou, X.; Zhao, Y.; Cheng, X.; and Wang, H. 2026. AgentFlow: Building Agent Dependency Graphs for Static Analysis of Agent Programs. *arXiv:2607.01640*.
- Wen, X.; Mo, W. J.; Xie, Y.; Qi, P.; and Chen, M. 2025. Towards Policy-Compliant Agents: Learning Efficient Guardrails for Policy Violation Detection. *arXiv:2510.03485*.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM)*.

Xue, Z.; Zhao, Y.; Wang, S.; Chen, K.; and Wang, H. 2025. A Characterization Study of Bugs in LLM Agent Workflow Orchestration Frameworks. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

Zhao, W.; Li, Z.; Zhang, P.; and Sun, J. 2026. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. arXiv:2604.11790.

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this .tex file directly.

For each question (that applies), replace the “Type your response here” text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
yes
```

Please make sure to:

- Replace **ONLY** the “Type your response here” text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the \question command or any other lines in this document.

You can \input this .tex file right before \end{document} of your main file or compile it as a stand-alone document. Check the instructions on your conference’s website to see if you will be asked to provide this checklist with your paper or separately.

1. General Paper Structure

- 1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) **yes**
- 1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) **yes**
- 1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) **yes**

2. Theoretical Contributions

- 2.1. Does this paper make theoretical contributions? (yes/no) **no**

If yes, please address the following points:

- 2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) **NA**
- 2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) **NA**
- 2.4. Proofs of all novel claims are included (yes/partial/no) **NA**
- 2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) **NA**
- 2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) **NA**
- 2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) **NA**
- 2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) **NA**

3. Dataset Usage

- 3.1. Does this paper rely on one or more datasets? (yes/no) **yes**

If yes, please address the following points:

- 3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) **yes**
- 3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) **partial**
- 3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) **yes**
- 3.5. All datasets drawn from the existing literature (potentially including authors’ own previously published work) are accompanied by appropriate cita-

tions (yes/no/NA) [NA](#)

- 3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) [NA](#)
- 3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisfying (yes/partial/no/NA) [NA](#)

4. Computational Experiments

- 4.1. Does this paper include computational experiments? (yes/no) [yes](#)

If yes, please address the following points:

- 4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) [NA](#)
- 4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) [yes](#)
- 4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) [yes](#)
- 4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) [yes](#)
- 4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) [partial](#)
- 4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) [yes](#)
- 4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) [partial](#)
- 4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) [yes](#)
- 4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) [yes](#)
- 4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, con-

fidence, or other distributional information (yes/no) [yes](#)

- 4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) [NA](#)
- 4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) [yes](#)